Coding e Fondamenti di Intelligenza Artificiale

A.A. 2026/27 — Lezione 12: Training e test

Vincenzo Nardelli



vincenzo.nardelli@unicatt.it · vincnardelli.com/cia

LAB Ha imparato davvero?

Nella lezione 11 l'albero ha trovato da solo le soglie dell'autovelox:
0,5 · 10,5 · 40,5 · 60,5, e la multa giusta per tutte le 5.000 auto.

Ma quelle 5.000 auto le aveva già viste.

Un modello si costruisce per usarlo su **casi nuovi**:
le auto che passeranno domani, i negozi che apriremo l'anno prossimo.
Come si misura se funzionerà su quelli?

LAB Training e test - Perché serve il test

L'esame con le domande dell'esercitazione

Uno studente che ha imparato a memoria le soluzioni prende il massimo.
Ha capito? Non lo sappiamo: bisogna fargli domande **nuove**.

Lo stesso vale per un modello

Misurarlo sugli esempi da cui ha imparato dice quanto li ricorda, non quanto ha capito.
Per saperlo servono esempi che **non ha mai visto**.

LAB Training e test



il modello impara da qui qui si misura quanto sbaglia

1. Si dividono i dati **prima** di cominciare, a caso.
2. Il modello impara solo dal **training**.
3. Il **test** non si tocca mentre il modello impara: serve solo alla fine, per misurare.
4. L'**accuracy** è la quota di risposte giuste. Quella che conta è sul test.

LAB Training e test - In Python

```
from sklearn.model_selection import train_test_split

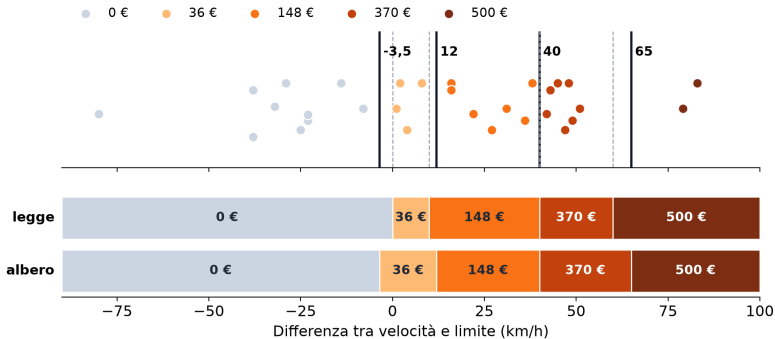
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=1)

albero = DecisionTreeClassifier(random_state=1)
albero.fit(X_train, y_train) # impara solo dal training

albero.score(X_train, y_train) # accuracy sul training
albero.score(X_test, y_test) # accuracy sul test: quella che conta
```

`random_state=1` fissa la divisione a caso: rieseguendo la cella, gli stessi esempi finiscono nel test.
`train_size=30` invece di `test_size`: 30 esempi nel training, tutti gli altri nel test.

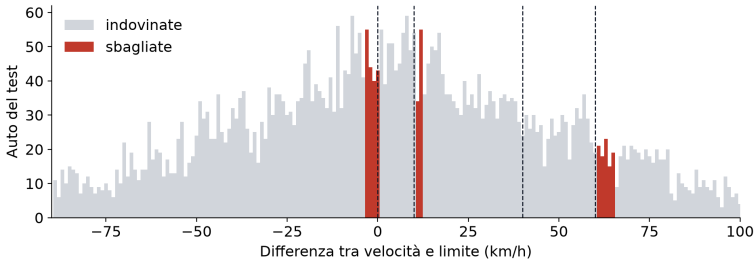
LAB Training e test - Le multe: 30 esempi



Accuracy sul **training**	100%
Accuracy sul **test**	92,6%

Soglie –3,5 · 12 · 40 · 65 invece di 0 · 10 · 40 · 60: con 30 esempi non poteva sapere dove passa il confine.

LAB Training e test - Le multe: dove sbaglia



Auto	Differenza	Multa vera	Prevista
182	da −3 a 0 km/h	0 €	36 €
89	11 e 12 km/h	148 €	36 €
96	da 61 a 65 km/h	500 €	370 €

Lontano dalle soglie non sbaglia mai: dove ha visto pochi esempi, indovina.

LAB Training e test - Le multe: più esempi

Stesso codice, cambia solo `train_size`:

Multe nel training	Training	Test	Soglie trovate
30	100%	92,6%	−3,5 · 12 · 40 · 65
500	100%	**100%**	0,5 · 10,5 · 40,5 · 60,5

Con 500 esempi le soglie sono quelle della legge.
Sul training è il 100% in tutti e due i casi: per questo l'accuracy sul training non dice niente.

LAB Training e test - I seggiolini per auto

seggiolini.csv: 400 negozi che vendono seggiolini per auto

negozio	vendite	prezzo	prezzo_conc.	pubblicita	scaffale	reddito	eta	…
1	9,50	120	138	11	cattivo	73	42	
2	11,22	83	111	16	buono	48	65	
3	10,06	80	113	10	medio	35	59	
4	7,40	97	117	4	medio	100	55	
…								

Domanda di business: guardando prezzo e pubblicità, sappiamo dire in anticipo quali negozi avranno **vendite alte** (più di 8.000 pezzi)?

Qui non c'è una regola esatta come per le multe: due negozi uguali possono vendere in modo diverso.

LAB Training e test - Seggiolini: il confronto

	Prezzo	Pubblicità
vendite basse (236)	122,6	5,0
vendite alte (164)	106,0	9,0

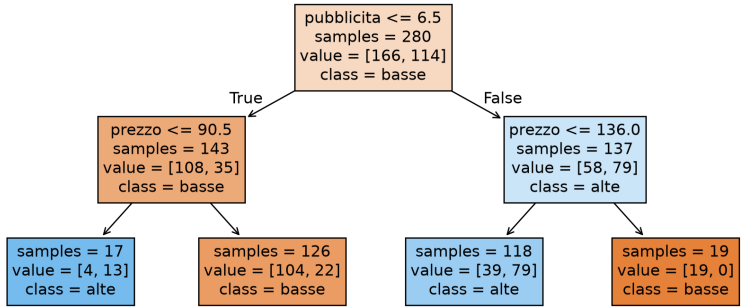
Chi vende molto ha in media prezzi più bassi e più pubblicità.

Dividiamo: **280** negozi nel training, **120** nel test.

Il modello più stupido dice sempre "vendite basse". Sul test indovina il **58,3%** (70 negozi su 120).

È la soglia da battere: un modello vale qualcosa solo se fa meglio.

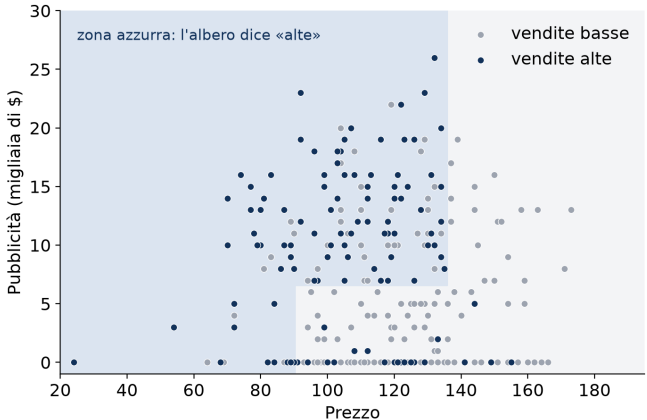
LAB Training e test - Seggiolini: l'albero



`max_depth=2`: al massimo due domande di fila. A ogni domanda, a sinistra se la risposta è sì.

Accuracy: **76,8%** sul training, **67,5%** sul test. Meno del training, come sempre, ma meglio del 58,3%.

LAB Training e test - Seggiolini: dove taglia



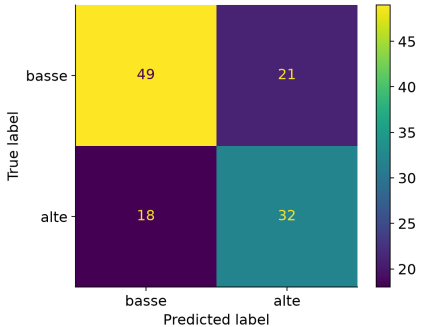
Ogni punto è un negozio del training.

Con poca pubblicità (fino a 6,5) servono prezzi bassissimi (fino a 90,5).

Con più pubblicità basta un prezzo non troppo alto (fino a 136).

In ogni zona ci sono punti dell'altro colore: sono gli errori.

LAB Training e test - La tabella di confusione



Negozi del test (120). Righe: la classe vera (*True label*); colonne: quella prevista (*Predicted label*).

- **49 + 32 = 81** giuste: il 67,5%
- **21** negozi con vendite basse dati per alti: se puntassimo su quelli, resteremmo delusi
- **18** negozi con vendite alte non riconosciuti: occasioni perse

L'accuracy dice *quanto* sbaglia, la tabella *come*.

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(albero_2, X_test, y_test,
                                      display_labels=["basse", "alte"])
```

LAB Training e test - Più profondo non vuol dire migliore

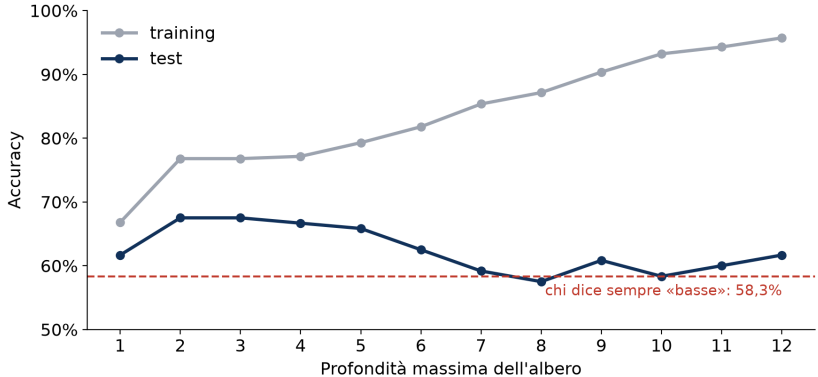
Stesso codice, cambia solo `max_depth`. Senza limiti l'albero fa tutte le domande che vuole.

Albero	Foglie	Training	Test
profondità 2	4	76,8%	**67,5%**
profondità 5	20	79,3%	65,8%
senza limiti	103	96,4%	60,8%

280 negozi e 103 foglie: due o tre negozi per foglia.
L'albero ha imparato a memoria il training, compresi i casi strani che nei negozi nuovi non si ripetono.

Si chiama **overfitting**: bravissimo sugli esempi visti, poco meglio di chi dice sempre "basse" su quelli nuovi.

LAB Training e test - Overfitting



Il training sale sempre: più domande, più negozi ricordati.
Il test arriva al 67,5% con profondità 2 e 3, poi scende. È il test a dirci dove fermarci.

LAB Training e test - Cosa abbiamo imparato

1. Un modello si giudica su dati che **non ha visto**: training e test si dividono prima di cominciare.
2. L'accuracy sul training è sempre ottimista; quella sul test dice come andrà sui casi nuovi.
3. Con **pochi esempi** le soglie sono approssimative (le multe).
4. Con **troppe domande** l'albero impara a memoria anche il rumore (i seggiolini): è l'overfitting.
5. Il primo confronto è con il modello più stupido. La tabella di confusione dice come sbaglia.

LAB - Vendite alte o basse?

`lab_seggiolini.ipynb`: una cella per domanda. Prima di eseguire, scrivete il risultato che vi aspettate.

1. Quanti negozi hanno vendite alte? Che quota dei 400 sono?
2. Qual è il reddito medio della zona per i negozi con vendite basse e alte?
3. Un albero con `max_depth=3` su prezzo e pubblicità: accuracy sul training e sul test?
4. Aggiungete la posizione sullo scaffale (`scaffale_num`) con `max_depth=2`: il test migliora rispetto al 67,5%?
5. La tabella di confusione di quell'albero: quanti negozi con vendite alte non riconosce?
6. Un negozio nuovo: prezzo 100, pubblicità 10, scaffale buono. Vendite alte o basse?

Extra: 7. con prezzo, pubblicità e scaffale, provate `max_depth=6` e `max_depth=10`: come cambiano training e test?

LAB - Correzione

1. 164 negozi su 400, il 41%.

2. 73,9 mila dollari dove le vendite sono alte, 65,0 dove sono basse.

3. Come con profondità 2: 76,8% e 67,5%. La terza domanda divide i gruppi ma non cambia nessuna previsione.

4. La prima domanda diventa “lo scaffale è buono?”. Training 77,9%, test **73,3%**: la variabile giusta vale più di tante domande in più.

5. 25 negozi con vendite alte su 50 dati per bassi: ne riconosce solo metà. Quelli con vendite basse li sbaglia di rado (7 su 70).

6. Vendite alte: scaffale buono e prezzo sotto 135.

7. Profondità 6: training 87,1%, test 77,5%. Profondità 10: training 95,4%, ma il test scende al 72,5%. Il training sale sempre; il test prima sale e poi scende.